Intermediate (two-level logger) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Where this exercise is saved: `practice/logging_practice.py`. Run it with `cd practice && python logging_practice.py`.

Used later by: the Real-world wiring exercise **copies** the `get_logger()` you write here into `practice/config_logging_wiring/logging_setup.py`, and the Build Task writes the same function again in `practice/build_task/logging_setup.py`. A copy, not an import — each of those folders stands alone. Keep the name and signature exactly as they are: `get_logger(name: str) -> logging.Logger`.

Story — `logging_practice.py`: `print()` gives you no way to separate "quiet, user-facing output" from "detailed diagnostic noise," and no record once the terminal scrolls away. `get_logger(name)` fixes both at once — quiet on screen, everything in a file. Written here, once, as its own exercise, because every later copy of this function (config_logging_wiring, Build Task, and beyond) builds on getting it right the first time. **If not:** the first time you'd wire up logging for real would be inside a bigger, more distracting file, with no smaller version to check your understanding against.

Solution

```
# practice/logging_practice.py
import logging

def get_logger(name: str) -> logging.Logger:
    # why: one function every file can call to get a logger that prints
    # INFO+ to the screen and DEBUG+ to a file.
    logger = logging.getLogger(name)
    # how: the logger's own level must be the lowest of any handler, or messages
    # get filtered here first
    logger.setLevel(logging.DEBUG)

    screen_handler = logging.StreamHandler()
    # when: keeps the terminal quiet — only INFO and above show up live
    screen_handler.setLevel(logging.INFO)
    logger.addHandler(screen_handler)

    file_handler = logging.FileHandler("app.log")
```

```
    # when: the file keeps everything, including DEBUG, for reading after
    # something breaks
    file_handler.setLevel(logging.DEBUG)
    logger.addHandler(file_handler)

    return logger

logger = get_logger(__name__)

logger.debug("This only goes to the file.")
logger.info("This goes to both the screen and the file.")
logger.warning("This also goes to both.")
```

Expected output (terminal):

```
This goes to both the screen and the file.
This also goes to both.
```

`app.log` after this runs:

```
This only goes to the file.
This goes to both the screen and the file.
This also goes to both.
```